Trabajo Práctico · Unidad 8

Food Store en la feria: la base de la tablet y el cierre de la capa de datos

Tecnicatura Universitaria en Programación · Bases de Datos II · Comisión 4
Integrante: Tiago Guillot · Repositorio: https://github.com/VIKROOM/DB-TPS-COM4
PostgreSQL 16 · SQLite 3.37 · MongoDB 7 (U7) · 04/11/2026

1. Qué datos viajan a la tablet (consigna 1.a)

La fuente de verdad es PostgreSQL. La tablet recibe solo una copia de lectura del catálogo mínimo para armar borradores sin señal. No viajan clientes, usuarios, contraseñas, ventas ni logs: no son necesarios para la venta offline y aumentarían el riesgo sobre datos personales. La tablet propone; el servidor dispone sobre precio, stock y ventas.

Campo que viaja	¿Viaja?	Por qué
producto.id / nombre	Sí	Identificar y mostrar el producto en el borrador.
precio_centavos	Sí	Precio visto para detectar cambios; entero, sin punto flotante.
stock_visto	Sí	Referencia para validar la cantidad; no descuenta stock.
disponible / eliminado	Sí	Semáforo y lápida de la baja lógica.
fecha_copia	Sí	Saber la antigüedad de la instantánea.
categoria_id / categoría	No	No se usa para armar el borrador.
cliente (id, nombre, email, teléfono)	No	Datos personales innecesarios en la tablet.
usuario / contraseña / roles	No	Seguridad: la tablet usa un vendedor inventado.
pedido / linea_pedido / ventas	No	Transaccional: lo decide PostgreSQL.
Logs / auditoría	No	No aportan a la operación offline.

La instantánea se proyecta con v_snapshot_movil: precio_centavos = round(precio × 100), disponible = activo AND stock > 0 y eliminado = NOT activo. El precio se convierte a centavos para que la comparación del semáforo sea exacta; la lápida de la baja lógica también viaja para que la tablet sepa que el producto existía y ya no está.

2. Sincronización y casos (consigna 2.c)

El circuito es: PostgreSQL exporta el CSV de v_snapshot_movil → SQLite lo carga en snapshot_servidor → un UPSERT refresca producto_cache sin borrar lápidas → el semáforo evalúa solo las solicitudes PENDIENTE. Para cada caso se preparó la situación en la copia del servidor, se exportó una instantánea nueva y se corrió el semáforo. La evidencia completa está en evidencias/04_semaforo_casos.txt.

Caso	Qué se preparó	Resultado	Explicación
C2-A	Borrador del producto 1 (2 × 105000). Se sube el precio a 1150,00 en el servidor.	CONFLICTO / PRECIO_CAMBIADO	No hay rojo, pero el precio visto difiere del actual. La solicitud queda en amarillo para que el cliente decida.
C2-B	Se acepta el precio (se actualiza a 115000) y se envía una solicitud nueva con otro UUID.	SIMULADO_APROBADO / TODO_COINCIDE	Al coincidir todos los datos, el borrador queda aprobado para simulación; no se confirma la venta.
C3	Borrador de 15 unidades del producto 4, con stock 10.	RECHAZADO / STOCK_INSUFICIENTE	La cantidad supera el stock visto: rojo, aunque el precio coincida.
C4	Borrador del producto 5 y luego UPDATE producto SET activo = FALSE WHERE id = 5.	RECHAZADO / PRODUCTO_DADO_DE_BAJA	La lápida viaja en la instantánea. El producto 5 ya no está disponible, pero su fila y el ítem del borrador siguen existiendo.
C5	Borrador del producto 2 (1 × 80000) sin cambios en el servidor.	SIMULADO_APROBADO / TODO_COINCIDE	Producto disponible, stock suficiente y precio igual: verde.
C6	Reenvío del mismo UUID y repetición del semáforo.	Error de PK; una sola fila	La PK de cola_sync rechaza el duplicado y repetir la simulación no reevalúa: evaluado_en no cambia.

Invariantes del servidor. Antes y después de toda la simulación: pedido = 3, linea_pedido = 5, producto = 5, cliente = 3, suma_stock = 115 e importe_lineas = $5.750,00. El diff está vacío: la tablet no descontó stock ni confirmó ventas (evidencias/05_verificacion_postgres_diff.txt). Lo que sí cambió es el precio del producto 1 (C2) y el flag activo del producto 5 (C4), porque son las preparaciones deliberadas de los casos.

Idempotencia en un sistema real (C6). La PK de SQLite solo protege a esa tablet. La protección real debe estar en el servidor: una clave única de idempotencia por solicitud, el registro de la respuesta y la confirmación dentro de una transacción. Si la tablet reintenta, el servidor devuelve el resultado ya registrado en lugar de crear una segunda venta.

3. Mapa de la capa de datos (consigna 3.a)

Motor	Qué guarda	Quién manda sobre esos datos
PostgreSQL 16	Fuente de verdad: categoria, producto (precio, stock, activo), cliente, pedido, linea_pedido, usuarios/roles, auditoría, vistas y la vista materializada de facturación.	PostgreSQL manda sobre precio, stock, ventas, clientes y seguridad. Ningún otro motor confirma una venta.
MongoDB 7 (U7)	Proyección documental de pedidos + detalle (12 pedidos / 24 líneas) para consultas de lectura. No guarda stock ni precios vigentes.	PostgreSQL manda. MongoDB es una copia de lectura; no acepta ventas ni modifica el catálogo.
SQLite 3.37+ (tablet)	producto_cache (catálogo de solo lectura), pedido_borrador, item_borrador y cola_sync (borradores y su estado).	La tablet no manda sobre nada: propone borradores. PostgreSQL dispone cuando confirma la venta.

El flujo es de una sola dirección: PostgreSQL → CSV → SQLite. La vuelta (borrador → servidor) queda fuera de este TP y, cuando exista, deberá confirmarse en PostgreSQL con una transacción y una clave de idempotencia. MongoDB no participa del circuito de venta: es una proyección de lectura de los pedidos ya confirmados.

4. Plan de mantenimiento (consigna 3.b)

¿Qué se respalda y cómo se prueba la restauración? Se respalda PostgreSQL completo con pg_dump -Fc (esquema + datos) y los roles con pg_dumpall -g. Se conservan 7 copias diarias y 4 semanales fuera del servidor, con sha256 para detectar corrupción. La restauración se prueba mensualmente en una base temporal: pg_restore --list, restauración completa y comparación de conteos, stock, importes y una consulta de muestra contra el original. El RPO objetivo es 24 h y el RTO se mide en cada simulacro (base del TP6).

Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets? No se reutiliza el catálogo local como si fuera actual. Al reconectar, la tablet descarta su snapshot_servidor y recibe una instantánea nueva; las solicitudes pendientes se reevalúan contra el estado restaurado y las que queden en CONFLICTO o RECHAZADO se derivan a revisión. Nunca se reenvían borradores automáticamente como ventas: si una venta confirmada se perdió con el respaldo, se reconcilia con la auditoría y se resuelve manualmente. El UUID evita duplicar una solicitud que ya había llegado.

¿Cada cuánto y quién revisa las solicitudes en CONFLICTO o RECHAZADO? El encargado de operaciones (rol de soporte) revisa la cola al cierre de cada feria y, como máximo, dentro de las 24 h. Si un caso sigue abierto a las 48 h, escala al responsable de datos. Cada decisión se registra en la auditoría y se resume semanalmente por causa (precio, stock, baja, producto inexistente) para corregir el origen.

5. Code review con IA (consigna 3.c)

Se revisaron los scripts de la cursada con OpenCode. Cada hallazgo se comprobó ejecutando el script real sobre PostgreSQL temporal, no por inspección visual. La salida completa está en evidencias/07_revision_codigo.txt; el runner es scripts/revision_codigo.py.

ID	Unidad	Hallazgo	Decisión	Cómo se comprobó
H1	U5	Los grants de tabla sobre usuario dan SELECT de todas las columnas (mail, celular, contraseña); el mínimo por columna solo se aplicó a rol_soporte.	Aceptado	role_table_grants vs. column_privileges: rol_app_lectura y rol_app_escritura tienen SELECT de tabla.
H2	U5	usuario_anon se crea con CTAS: no tiene PK ni NOT NULL y la verificación por EXCEPT no detecta duplicados.	Aceptado	pg_constraint = 0; un INSERT duplicado de id pasó y diff_total siguió en 0.
H3	U3	v_pedidos_cliente filtra c.activo = TRUE y oculta los pedidos históricos de clientes inactivos.	Aceptado	2 pedidos en la tabla vs. 1 en la vista; el pedido 2 (cliente inactivo) desaparece.
H4	U3/U5	U5 referencia usuario/detalle_pedido; el esquema real de U3 usa cliente/linea_pedido.	Aceptado	to_regclass devuelve NULL para usuario y detalle_pedido; el GRANT falla con “relation does not exist”.
H5	U3	registrar_pedido consume un id de secuencia antes de validar los ítems; se sospechaban pedidos huérfanos.	Descartado	Secuencia 2 → 3, pero pedido sigue en 2 y hay 0 huérfanos: RAISE EXCEPTION aborta la transacción. Los huecos de secuencia son normales.
H6	U5	ALTER DEFAULT PRIVILEGES ... GRANT SELECT (columna) no es válido y corta el script de U5.	Aceptado	Error “default privileges cannot be set for columns”; pg_default_acl solo registra privilegios de tabla.

Desarrollo de los hallazgos principales

H3 (U3). La vista v_pedidos_cliente agrega WHERE c.activo = TRUE, una decisión razonable para clientes vigentes pero incorrecta para un historial: un cliente que se dio de baja conserva sus pedidos. La prueba insertó 3 clientes (uno inactivo) y 2 pedidos; la vista devolvió 1 y ocultó el pedido 2. Decisión aceptada: el historial debe incluir a todos los clientes y, si se necesita el filtro, aplicarlo en otra vista o en la consulta, no en la fuente.

H4 (U3/U5). Los scripts de U5 se escribieron contra nombres que no existen en el esquema final: usuario y detalle_pedido. En U3 las tablas son cliente y linea_pedido. El GRANT de U5 falla al ejecutarse sobre U3, así que el script de seguridad no es reproducible tal cual. Decisión aceptada: unificar nombres o agregar una capa de vistas compatibles antes de aplicar los permisos.

H5 (U3), descartado. La sospecha era que registrar_pedido consumía la secuencia antes de validar y dejaba un pedido huérfano. La prueba con un producto inexistente mostró que la secuencia avanza de 2 a 3, pero la tabla pedido sigue con 2 filas y la consulta de huérfanos devuelve 0: la excepción aborta la transacción. Se descarta como defecto. Los huecos de secuencia no afectan integridad ni negocio y son esperables.

H6 (U5). PostgreSQL no permite ALTER DEFAULT PRIVILEGES con lista de columnas; el script se corta en esa línea. La consulta a pg_default_acl confirma que solo se registraron los privilegios de tabla. Decisión aceptada: quitar las columnas del default privilege y mantener el grant por columna sobre la tabla existente, o crear una vista usuario_soporte y otorgar SELECT sobre la vista.

6. Evidencias y reproducción

python scripts/simular_tp8.py reproduce las consignas 1 y 2 y escribe evidencias/00 a 06; python scripts/revision_codigo.py reproduce la consigna 3.c en evidencias/07_revision_codigo.txt. Los datos son inventados y la copia de PostgreSQL es de laboratorio. El detalle del orden de ejecución está en README.md; la especificación previa, en docs/kiro/; y el uso de IA, en docs/bitacora_ia.md y docs/DUIA.md.